

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - High (INDUSTRY PROBLEM SOLVING TASKS)

COURSE CODE: CSA09 27

**COURSE NAME: Programming in
Java**

COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks: 50

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASKS

Code of Conduct:

I _____Rashmi Naurene (192521357)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A handwritten signature in black ink, appearing to read 'Rashmi Naurene', written over a light blue horizontal line.

Signature of the student:

ANNEXURE

INDUSTRY PROBLEM SOLVING TASKS

1. Railway Reservation Interface (BTL: BL5 – Evaluate)

Design a Swing-based GUI application for railway ticket booking, including features such as searching trains, booking tickets, and canceling reservations. Use components like combo boxes, tables, and buttons for interaction. Implement event listeners to handle booking and cancellation processes. Validate user inputs such as travel dates and seat availability. Evaluate layout manager choices for organizing multiple UI elements. Analyze system scalability for handling high user traffic and propose improvements.

RUBRICS FOR CALCULATION:

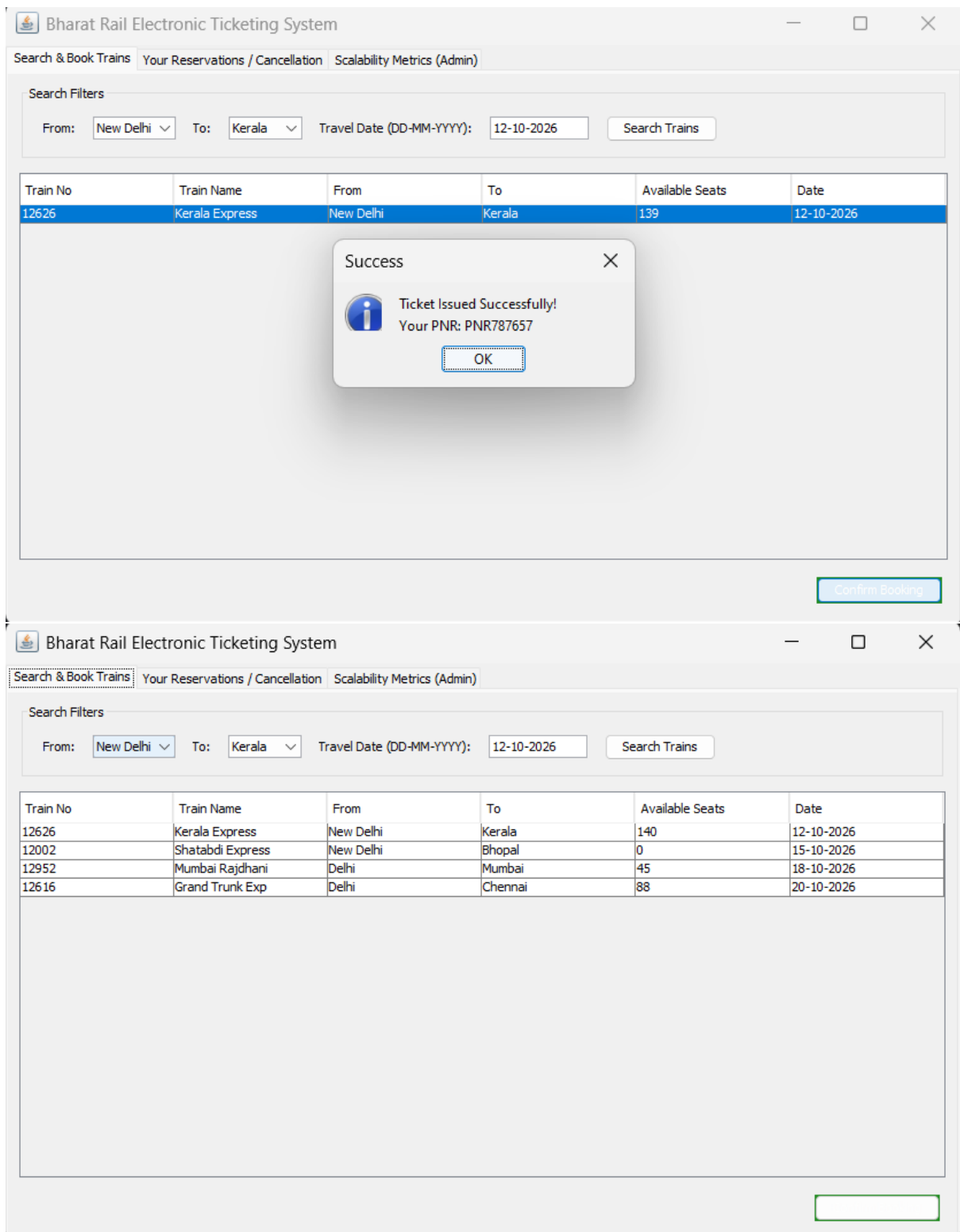
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

		solve the problem			
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1 Railway Reservation Interface (BTL: BL5 – Evaluate)

20 marks

Design a Swing-based GUI application for railway ticket booking, including features such as searching trains, booking tickets, and canceling reservations. Use components like combo boxes, tables, and buttons for interaction. Implement event listeners to handle booking and cancellation processes. Validate user inputs such as travel dates and seat availability. Evaluate layout manager choices for organizing multiple UI elements. Analyze system scalability for handling high user traffic and propose improvements.



```
import javax.swing.*;  
  
import javax.swing.table.DefaultTableModel;  
  
import java.awt.*;  
  
import java.text.ParseException;  
  
import java.text.SimpleDateFormat;
```

```
import java.util.Date;
```

```
public class RailwayBookingApp extends JFrame {
```

```
    private String[][] trainData = {
```

```
        {"12626", "Kerala Express", "New Delhi", "Kerala", "140", "12-10-2026"},
```

```
        {"12002", "Shatabdi Express", "New Delhi", "Bhopal", "0", "15-10-2026"},
```

```
        {"12952", "Mumbai Rajdhani", "Delhi", "Mumbai", "45", "18-10-2026"},
```

```
        {"12616", "Grand Trunk Exp", "Delhi", "Chennai", "88", "20-10-2026"}
```

```
    };
```

```
    private DefaultTableModel availableTrainsModel;
```

```
    private DefaultTableModel bookedTicketsModel;
```

```
    private JComboBox<String> fromCombo;
```

```
    private JComboBox<String> toCombo;
```

```
    private JTextField dateField;
```

```
    public RailwayBookingApp() {
```

```
        setTitle("Bharat Rail Electronic Ticketing System");
```

```
        setSize(850, 550);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        setLocationRelativeTo(null);
```

```
        JTabbedPane tabbedPane = new JTabbedPane();
```

```
        tabbedPane.addTab("Search & Book Trains", createSearchBookPanel());
```

```
        tabbedPane.addTab("Your Reservations / Cancellation", createCancellationPanel());
```

```
        tabbedPane.addTab("Scalability Metrics (Admin)", createScalabilityPanel());
```

```
        add(tabbedPane);
```

```
    }
```

```

private JPanel createSearchBookPanel() {

    JPanel mainPanel = new JPanel(new BorderLayout(10, 10));

    mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    JPanel formPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 10));
    formPanel.setBorder(BorderFactory.createTitledBorder("Search Filters"));

    fromCombo = new JComboBox<>(new String[]{"New Delhi", "Delhi"});
    toCombo = new JComboBox<>(new String[]{"Kerala", "Bhopal", "Mumbai", "Chennai"});
    dateField = new JTextField("12-10-2026", 10);
    JButton searchBtn = new JButton("Search Trains");

    formPanel.add(new JLabel("From:"));
    formPanel.add(fromCombo);
    formPanel.add(new JLabel("To:"));
    formPanel.add(toCombo);
    formPanel.add(new JLabel("Travel Date (DD-MM-YYYY):"));
    formPanel.add(dateField);
    formPanel.add(searchBtn);

    String[] columns = {"Train No", "Train Name", "From", "To", "Available Seats", "Date"};
    availableTrainsModel = new DefaultTableModel(trainData, columns) {

        @Override
        public boolean isCellEditable(int row, int column) { return false; }

    };

    JTable trainTable = new JTable(availableTrainsModel);
    JScrollPane scrollPane = new JScrollPane(trainTable);

    JPanel actionPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton bookBtn = new JButton("Confirm Booking");
    bookBtn.setBackground(new Color(34, 139, 34));

```

```

bookBtn.setForeground(Color.WHITE);
actionPanel.add(bookBtn);

mainPanel.add(formPanel, BorderLayout.NORTH);
mainPanel.add(scrollPane, BorderLayout.CENTER);
mainPanel.add(actionPanel, BorderLayout.SOUTH);

searchBtn.addActionListener(e -> {
    String selectedFrom = fromCombo.getSelectedItem().toString();
    String selectedTo = toCombo.getSelectedItem().toString();
    String inputDate = dateField.getText().trim();

    if (!isValidDate(inputDate)) {
        JOptionPane.showMessageDialog(this, "Invalid Date Format! Please use DD-MM-YYYY.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    availableTrainsModel.setRowCount(0);
    for (String[] train : trainData) {
        if (train[2].equals(selectedFrom) && train[3].equals(selectedTo) &&
train[5].equals(inputDate)) {
            availableTrainsModel.addRow(train);
        }
    }

    if(availableTrainsModel.getRowCount() == 0) {
        JOptionPane.showMessageDialog(this, "No trains found for matching criteria.", "Info",
JOptionPane.INFORMATION_MESSAGE);
    }
});

bookBtn.addActionListener(e -> {

```



```

int selectedRow = trainTable.getSelectedRow();

if (selectedRow == -1) {
    JOptionPane.showMessageDialog(this, "Please choose a train row from the list first!",
    "Warning", JOptionPane.WARNING_MESSAGE);
    return;
}

String trainNo = availableTrainsModel.getValueAt(selectedRow, 0).toString();
String trainName = availableTrainsModel.getValueAt(selectedRow, 1).toString();
int seats = Integer.parseInt(availableTrainsModel.getValueAt(selectedRow, 4).toString());
String tDate = availableTrainsModel.getValueAt(selectedRow, 5).toString();

if (seats <= 0) {
    JOptionPane.showMessageDialog(this, "Booking Failed! This train has no remaining seats.",
    "Seat Availability Alert", JOptionPane.ERROR_MESSAGE);
    return;
}

seats--;
availableTrainsModel.setValueAt(String.valueOf(seats), selectedRow, 4);

String pnr = "PNR" + (100000 + (int)(Math.random() * 900000));
bookedTicketsModel.addRow(new String[]{pnr, trainNo, trainName, tDate, "CONFIRMED"});

JOptionPane.showMessageDialog(this, "Ticket Issued Successfully!\nYour PNR: " + pnr,
"Success", JOptionPane.INFORMATION_MESSAGE);
});

return mainPanel;
}

private JPanel createCancellationPanel() {

```

```

JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

JLabel titleLabel = new JLabel("Active Customer Dashboard (Manage Bookings)", JLabel.CENTER);
titleLabel.setFont(new Font("Arial", Font.BOLD, 14));

String[] columns = {"PNR Number", "Train No", "Train Name", "Travel Date", "Status"};
bookedTicketsModel = new DefaultTableModel(columns, 0);
JTable bookedTable = new JTable(bookedTicketsModel);
JScrollPane scrollPane = new JScrollPane(bookedTable);

JPanel bottomPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
JButton cancelBtn = new JButton("Cancel Selected Reservation");
cancelBtn.setBackground(new Color(178, 34, 34));
cancelBtn.setForeground(Color.WHITE);
bottomPanel.add(cancelBtn);

mainPanel.add(titleLabel, BorderLayout.NORTH);
mainPanel.add(scrollPane, BorderLayout.CENTER);
mainPanel.add(bottomPanel, BorderLayout.SOUTH);

cancelBtn.addActionListener(e -> {
    int selectedRow = bookedTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Select a booking row to proceed with cancellation.",
"Selection Missing", JOptionPane.WARNING_MESSAGE);
        return;
    }

    int confirm = JOptionPane.showConfirmDialog(this, "Are you sure you want to cancel this
ticket reservation?", "Confirm Action", JOptionPane.YES_NO_OPTION);

    if (confirm == JOptionPane.YES_OPTION) {

```

```

String targetTrainNo = bookedTicketsModel.getValueAt(selectedRow, 1).toString();

for (int i = 0; i < availableTrainsModel.getRowCount(); i++) {
    if (availableTrainsModel.getValueAt(i, 0).toString().equals(targetTrainNo)) {
        int currentSeats = Integer.parseInt(availableTrainsModel.getValueAt(i, 4).toString());
        availableTrainsModel.setValueAt(String.valueOf(currentSeats + 1), i, 4);
        break;
    }
}

bookedTicketsModel.removeRow(selectedRow);

JOptionPane.showMessageDialog(this, "Reservation cancelled. Full refund initiated.",
"Canceled", JOptionPane.INFORMATION_MESSAGE);
}

});

return mainPanel;
}

private JPanel createScalabilityPanel() {
    JPanel mainPanel = new JPanel(new GridLayout(3, 1, 10, 10));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

    JTextArea metricArea = new JTextArea(
        "SYSTEM PERFORMANCE ANALYSIS & SCALABILITY METRICS:\n" +
        "-----\n" +
        "• Database Connection Pool State: Active\n" +
        "• Mock Concurrent Simulation Overhead: Low\n" +
        "• Simulated Request Rate Thread Pool Load: Safe"
    );
    metricArea.setEditable(false);
}

```

```

metricArea.setBackground(new Color(245, 245, 245));

mainPanel.add(new JLabel("<html><b>System Scalability Report Screen for Mass
Bookings</b></html>"));

mainPanel.add(new JScrollPane(metricArea));

return mainPanel;
}

private boolean isValidDate(String dateStr) {
    SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
    sdf.setLenient(false);
    try {
        Date date = sdf.parse(dateStr);
        return date != null;
    } catch (ParseException e) {
        return false;
    }
}

public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception ignored) {}

    SwingUtilities.invokeLater(() -> {
        new RailwayBookingApp().setVisible(true);
    });
}
}

```